



■ Online Book Store Management System

12-Week Development Roadmap
Full Stack · Database · Software Engineering

Project Type	Full Stack Web Application
Duration	12 Weeks
Department	Computer Science & Engineering
Technology	PostgreSQL · Node.js · React · Git

Samia Sikder
Roll: 251035014

Version 1.0 | June 2026

1. Project Overview

Platform	Online Book Store Management System — Full Stack Web App
Goal	Enable customers to browse, order, and review books; give admins complete control over inventory, orders & analytics
Type	Full Stack · Database · Software Engineering
Duration	12 Weeks (June – August 2026)
Stack	PostgreSQL · Node.js / Express · React · MongoDB · Git/GitHub · REST API

2. Core Objectives

User Auth	Signup/Login, JWT sessions, bcrypt password hashing, role-based access (Customer / Admin)
Book Catalogue	Browse, search, filter by category/author; paginated listing with cover images
Shopping Cart	Add/remove items, quantity control, cart persistence across sessions
Order System	Place orders, payment simulation, order history with status tracking
Admin Panel	CRUD for books/authors/categories, inventory management, order fulfilment
Reviews	Star ratings + text reviews per book; stored in MongoDB for flexible schema
Analytics	Monthly sales, top customers, revenue trends — visualised in admin dashboard

3. System Architecture



Figure 1: Customer → React Frontend → Express REST API → PostgreSQL (relational) + MongoDB (reviews/logs)

4. REST API Endpoints

Method	Endpoint	Description
POST	/register	Register a new customer account
POST	/login	Authenticate and receive JWT token
GET	/books	List all books with filters & pagination
GET	/books/:id	Get single book details
POST	/books	Admin: add a new book
PUT	/books/:id	Admin: update book details
DELETE	/books/:id	Admin: remove a book
GET	/cart	Retrieve current user's cart
POST	/cart	Add item to cart
POST	/orders	Place an order from cart
GET	/orders/:id	Get order details and status
GET	/reviews/:bookId	Get reviews for a book
POST	/reviews	Submit a new review
GET	/admin/stats	Admin-only: platform analytics

5. Database Schema

Users	User_ID (PK) Name Email (UNIQUE) Password (hashed) Role (customer/admin) Created_At
Books	Book_ID (PK) Title Author_ID (FK) Category_ID (FK) Price Stock ISBN Cover_URL
Authors	Author_ID (PK) Name Bio
Categories	Category_ID (PK) Name Description
Orders	Order_ID (PK) User_ID (FK) Total_Amount Status Created_At
Order_Items	Item_ID (PK) Order_ID (FK) Book_ID (FK) Quantity Price_At_Purchase

Note: Book Reviews stored in MongoDB for flexible nested schema (rating, text, likes, timestamps).

6. 12-Week Sprint Timeline

Week	Focus Area	Key Deliverables
W1	Requirements & Planning	SRS Document, User Stories, Use Cases, Data Flow Diagrams
W2	ER Modeling	ER Diagram (7 entities), Relationships, PK/FK, Constraints
W3	Normalization & Schema	3NF Schema, PostgreSQL Setup, DDL Scripts, GitHub Repo
W4	Advanced SQL	JOIN Queries, Subqueries, Aggregations, Sample Data
W5	Transactions & Triggers	Order Transactions, COMMIT/ROLLBACK, Stock Triggers
W6	REST API & SE Design	Express Backend, Auth + Books + Orders APIs, MVC Structure

W7	Indexing & Performance	Indexes on Title/Category/UserID, EXPLAIN ANALYZE Report
W8	Stored Procedures & Views	Place Order Procedure, Invoice Generator, Best-Sellers View
W9	NoSQL & Data Import	MongoDB Reviews, Activity Logs, CSV Book Import
W10	Frontend Development	React UI: Homepage, Book Detail, Cart, Checkout, Admin
W11	Testing & Security	Unit + Integration Tests, SQL Injection Prevention, Roles
W12	Final Polish & Deployment	Analytics Dashboard, Documentation, Live Demo, Viva Prep

7. Detailed Phase Roadmap

1 — Foundation & Design (Weeks 1-3)

■ Week 1 Requirements & Planning SE

- Define features: signup/login, browse books, cart, orders, reviews
- Identify actors: Customer, Admin
- Write SRS Document — User Stories, Use Cases, Data Flow Diagrams

■ Week 2 ER Modeling DB · LAB

- Design ER Diagram with 7 entities: Users, Books, Authors, Categories, Orders, Order_Items, Reviews
- Define Primary & Foreign Keys, 1:M and M:N Relationships
- Add Constraints (NOT NULL, UNIQUE, CHECK)

■ Week 3 Normalization & Schema DB · LAB

- Normalize to 3NF — eliminate partial & transitive dependencies
- Convert ER → Relational Schema
- Setup PostgreSQL, create tables with DDL scripts

■ Checkpoint 1 — End of Week 3

- ✓ SRS Document
- ✓ ER Diagram
- ✓ Normalized Schema
- ✓ PostgreSQL DDL Scripts
- ✓ GitHub Repo with README

2 — SQL & Backend Core (Weeks 4-6)

■ Week 4 Advanced SQL DB · LAB

- Write JOIN queries (books + authors + categories)
- Subqueries for top-selling books and user order history
- Aggregations: total sales per category, average ratings; insert realistic sample data

**■ Week
5****Transactions & Integrity**

DB · LAB

- Implement order placement transaction with BEGIN / COMMIT / ROLLBACK
- Simulate payment flow with error handling
- Add triggers: reduce stock after order, prevent negative inventory

**■ Week
6****REST API & SE Design**

SE · LAB

- Build backend using Node.js / Express with MVC structure
- API endpoints: Auth (login/signup), Books (CRUD), Orders, Cart
- Document API with Swagger; test all endpoints in Postman

■ Checkpoint 2 (Midterm) — Week 6

- ✓ Live SQL queries demo
- ✓ 3+ working API endpoints
- ✓ Transaction rollback demo
- ✓ Viva on DB concepts

Optimization & Advanced DB

■ Week 7 Indexing & Performance

DB · LAB

- Add indexes on: book title, category, user_id
- Use EXPLAIN ANALYZE to profile slow queries
- Produce before vs after performance comparison report

■ Week 8 Stored Procedures & Views

DB · LAB

- Stored procedures: Place Order, Generate Invoice
- Views: Best-Selling Books, Monthly Revenue, Top Customers
- Add error handling and EXCEPTION blocks

■ Week 9 NoSQL & Data Import

DB · LAB

- Use MongoDB for book reviews and user activity logs
- Import CSV dataset (books list) into PostgreSQL
- Compare SQL vs NoSQL use-cases and document findings

■ Checkpoint 3 — Week 9

- ✓ Performance report (before/after)
- ✓ Stored procedures & views
- ✓ MongoDB integration
- ✓ CSV import script
- ✓ Updated GitHub

— Frontend, Testing & SE (We

■ Week 10 Frontend & SDLC

SE · LAB

- Build React UI: Homepage, Book Details, Cart & Checkout, Admin Dashboard
- Apply Agile methodology — sprint planning and retrospectives
- Git branching strategy: feature branches, pull requests, code review

■ Week 11 Testing & Security

SE · DB · LAB

- Unit testing for all API endpoints
- Integration testing for auth + order flow
- Input validation, SQL injection prevention, role-based route protection

■ Checkpoint 4 — Week 11

- ✓ Full working application
- ✓ Test report (unit + integration)
- ✓ Security checklist
- ✓ Git workflow proof (branches, PRs)

— Final Polish & Presentation

■ Week 12

Final Demo & Documentation

DB · SE · LAB

- Add analytics dashboard: monthly sales, top customers, revenue trends
- Write final project documentation and academic report
- Prepare live demo and viva presentation

■ ★ Final Evaluation — Week 12

- ✓ Live demo of complete application
- ✓ Viva from DB, SE, and Lab subjects
- ✓ Full code on GitHub (with README)
- ✓ Final project report
- ✓ Peer evaluation form

8. Technology Stack

Backend & API	Database	Frontend & Tools
Node.js Core runtime	PostgreSQL Relational DB	React UI library
Express.js REST framework	pgAdmin GUI tool	HTML/CSS/JS Styling
JWT Auth tokens	MongoDB Reviews/logs	Postman API testing
bcrypt Password hashing	SQL Schema & queries	GitHub Version control

9. Suggested Grading

Milestone	Weight	Covers
Checkpoint 1 — SRS + Schema	20%	Requirements, ER Diagram, Normalized SQL
Checkpoint 2 — Midterm	25%	SQL Queries, REST APIs, Transactions Demo
Checkpoint 3 — Optimization	15%	Indexing, Stored Procedures, MongoDB
Checkpoint 4 — Testing + UI	15%	React Frontend, Test Report, Security
Final Demo + Viva	25%	Live Application, Presentation, Peer Evaluation

10. Target Outcomes After 12 Weeks

- Full Stack project with React frontend + Express backend — live and accessible in browser
- REST API development with Node.js/Express — all endpoints documented and Postman-tested
- Secure authentication — JWT login, bcrypt hashing, role-based access control
- Relational database design & implementation — normalized PostgreSQL schema with ER diagram
- NoSQL integration — MongoDB for reviews and activity logs
- Advanced SQL — transactions, triggers, stored procedures, views, indexing
- Software Engineering practices — Agile, Git branching, MVC architecture, testing
- Strong portfolio project for web development and database engineering interviews